

```
package com.mycompany.message;
```

```
import java.util.ArrayList;
```

```
import java.util.Random;
```

```
import java.util.Scanner;
```

```
public class Message {
```

```
    // PART 1 - LOGIN FIELDS
```

```
    private String username;
```

```
    private String password;
```

```
    private String cellPhoneNumber;
```

```
    private String firstName;
```

```
    private String lastName;
```

```
    // PART 2 - MESSAGE FIELDS
```

```
    private String messageID;
```

```
    private String recipient;
```

```
    private String message;
```

```
    private int numMessagesSent;
```

```
    private String messageHash;
```

```
    private static int totalMessagesSent = 0;
```

```
    private static ArrayList<String> sentMessages = new ArrayList<>();
```

```
    private static ArrayList<String> messageHashes = new ArrayList<>();
```

```
    private static ArrayList<String> messageIDs = new ArrayList<>();
```

```
// RESET METHOD (important for JUnit tests)
```

```
public static void resetData() {
```

```
    sentMessages.clear();
```

```
    messageHashes.clear();
```

```
    messageIDs.clear();
```

```
    totalMessagesSent = 0;
```

```
}
```

```
// CONSTRUCTOR (PART 1)
```

```
public Message(String username, String password, String cellPhoneNumber, String  
firstName, String lastName) {
```

```
    this.username = username;
```

```
    this.password = password;
```

```
    this.cellPhoneNumber = cellPhoneNumber;
```

```
    this.firstName = firstName;
```

```
    this.lastName = lastName;
```

```
}
```

```
// CONSTRUCTOR (PART 2)
```

```
public Message(String recipient, String message) {
```

```
    this.recipient = recipient;
```

```
    this.message = message;
```

```
    this.messageID = generateMessageID();
```

```
    this.numMessagesSent = ++totalMessagesSent;
```

```
    this.messageHash = createMessageHash();
```

```
}
```

```
// USERNAME CHECK
```

```
public boolean checkUserName() {  
    return username.contains("_") && username.length() <= 5;  
}
```

```
// PASSWORD CHECK
```

```
public boolean checkPasswordComplexity() {  
    if (password.length() < 8) return false;  
  
    boolean hasUpper = false, hasDigit = false, hasSpecial = false;  
  
    for (char c : password.toCharArray()) {  
        if (Character.isUpperCase(c)) hasUpper = true;  
        else if (Character.isDigit(c)) hasDigit = true;  
        else if (!Character.isLetterOrDigit(c)) hasSpecial = true;  
    }  
    return hasUpper && hasDigit && hasSpecial;  
}
```

```
// PHONE CHECK
```

```
public boolean checkCellPhoneNumber() {  
    return cellPhoneNumber.matches("^\\+\\d{1,12}$");  
}
```

```
// REGISTER
```

```
public String registerUser() {  
    if (!checkUserName()) return "Username is not correctly formatted";  
    if (!checkPasswordComplexity()) return "Password is not correctly formatted";  
    if (!checkCellPhoneNumber()) return "Cell phone number is incorrectly formatted";  
  
    return "User registered successfully!";  
}
```

```
// LOGIN
```

```
public boolean loginUser(String enteredUsername, String enteredPassword) {  
    return username.equals(enteredUsername) && password.equals(enteredPassword);  
}
```

```
public String returnLoginStatus(boolean success) {  
    if (success) {  
        return "Welcome " + firstName + " " + lastName + ", it is great to see you again.";  
    }  
    return "Username or password incorrect, please try again.";  
}
```

```
// MESSAGE ID
```

```
private String generateMessageID() {  
    Random rand = new Random();  
    long id = (long) (rand.nextDouble() * 9_000_000_000L) + 1_000_000_000L;  
    return String.valueOf(id);  
}
```

```
}
```

```
// FIXED VALIDATION
```

```
public boolean checkMessageID() {  
    return messageID != null && messageID.matches("\\d{10}");  
}
```

```
// RECIPIENT CHECK
```

```
public String checkRecipientCell() {  
    if (recipient.matches("^\\+\\d{1,12}$")) {  
        return "Cell phone number successfully captured.";  
    }  
  
    return "Cell phone number is incorrectly formatted or does not contain an international  
code. Please correct the number and try again.";  
}
```

```
// HASH
```

```
public String createMessageHash() {  
    String[] words = message.trim().split("\\s+");  
    String firstWord = words[0].replaceAll("[^a-zA-Z0-9]", "");  
    String lastWord = words[words.length - 1].replaceAll("[^a-zA-Z0-9]", "");  
  
    return (messageID.substring(0, 2) + ":" + (numMessagesSent - 1) + ":" +  
        firstWord + lastWord).toUpperCase();  
}
```

```
// FIXED METHOD NAME

public String sendMessage(Scanner scanner) {

    System.out.println("\n1) Send");
    System.out.println("2) Disregard");
    System.out.println("3) Store");
    System.out.print("Choose: ");

    String choice = scanner.nextLine();

    switch (choice) {
        case "1":
            sentMessages.add(message);
            messageHashes.add(messageHash);
            messageIDs.add(messageID);
            return "Message successfully sent.";

        case "2":
            return "Message disregarded.";

        case "3":
            return "Message stored.";

        default:
            return "Invalid option.";
    }
}
```

```
}
```

```
// PRINT MESSAGES
```

```
public static String printMessages() {
```

```
    if (sentMessages.isEmpty()) return "No messages sent.";
```

```
    StringBuilder sb = new StringBuilder();
```

```
    for (int i = 0; i < sentMessages.size(); i++) {
```

```
        sb.append(i + 1).append(" ").append(sentMessages.get(i)).append("\n");
```

```
    }
```

```
    return sb.toString();
```

```
}
```

```
// TOTAL MESSAGES
```

```
public static int returnTotalMessages() {
```

```
    return totalMessagesSent;
```

```
}
```

```
// DISPLAY
```

```
public void displayMessageDetails() {
```

```
    System.out.println("\nMessage Details");
```

```
    System.out.println("ID: " + messageId);
```

```
    System.out.println("Hash: " + messageHash);
```

```
    System.out.println("Recipient: " + recipient);
```

```
    System.out.println("Message: " + message);
```

```
}
```

```
// GETTERS  
public String getMessage() { return message; }  
public String getMessageHash() { return messageHash; }  
}
```

```
package com.mycompany.message;
```

```
import org.junit.jupiter.api.Test;
```

```
import static org.junit.jupiter.api.Assertions.*;
```

```
public class MessageTest {
```

```
    @Test
```

```
    public void testSetup() {
```

```
        Message.resetData();
```

```
    }
```

```
    @Test
```

```
    public void testUsernameCorrect() {
```

```
        Message user = new Message("kyl_1", "Pass@123", "+2783", "Kyle", "Smith");
```

```
        assertTrue(user.checkUserName());
```

```
    }
```

```
    @Test
```

```
    public void testUsernameWrong() {
```

```
        Message user = new Message("kyle!!!!", "Pass@123", "+2783", "Kyle", "Smith");
```

```
        assertFalse(user.checkUserName());
```

```
    }
```

```
    @Test
```

```
    public void testPasswordValid() {
```

```
Message user = new Message("kyl_1", "Pass@123", "+2783", "Kyle", "Smith");
assertTrue(user.checkPasswordComplexity());
}
```

@Test

```
public void testPasswordInvalid() {
    Message user = new Message("kyl_1", "password", "+2783", "Kyle", "Smith");
    assertFalse(user.checkPasswordComplexity());
}
```

@Test

```
public void testPhoneValid() {
    Message user = new Message("kyl_1", "Pass@123", "+27838968976", "Kyle", "Smith");
    assertTrue(user.checkCellPhoneNumber());
}
```

@Test

```
public void testPhoneInvalid() {
    Message user = new Message("kyl_1", "Pass@123", "083", "Kyle", "Smith");
    assertFalse(user.checkCellPhoneNumber());
}
```

@Test

```
public void testLoginSuccess() {
    Message user = new Message("kyl_1", "Pass@123", "+2783", "Kyle", "Smith");
    assertTrue(user.loginUser("kyl_1", "Pass@123"));
}
```

```
}
```

```
@Test
```

```
public void testLoginFail() {
```

```
    Message user = new Message("kyl_1", "Pass@123", "+2783", "Kyle", "Smith");
```

```
    assertFalse(user.loginUser("kyl_1", "wrong"));
```

```
}
```

```
@Test
```

```
public void testMessageID() {
```

```
    Message msg = new Message("+2783", "Hello world message test");
```

```
    assertTrue(msg.checkMessageID());
```

```
}
```

```
@Test
```

```
public void testRecipient() {
```

```
    Message msg = new Message("+2783", "Hello world");
```

```
    assertEquals("Cell phone number successfully captured.", msg.checkRecipientCell());
```

```
}
```

```
@Test
```

```
public void testHashContains() {
```

```
    Message msg = new Message("+2783", "Hello world tonight");
```

```
    assertTrue(msg.getMessageHash().contains("HELLOTONIGHT"));
```

```
}
```

```
@Test
public void testTotalMessages() {
    Message.resetData();

    int before = Message.returnTotalMessages();
    new Message("+2783", "Msg1");
    new Message("+2784", "Msg2");

    int after = Message.returnTotalMessages();

    assertTrue(after >= before + 2);
}
}
```

Oracle. Java Documentation. Available at: <https://docs.oracle.com/javase/> [Accessed 25 May 2026].

Baeldung. Java Regex Phone Number Validation. Available at:
<https://www.baeldung.com/java-regex-validate-phone-numbers> [Accessed 25 May 2026].

OpenAI. ChatGPT. Available at: <https://chat.openai.com/> [Accessed 25 May 2026].